SİVAS
BİLİM VE TEKNOLOJİ
ÜNİVERSİTESİ

COMPUTER ENGINEERING

REPORT OF MODULE PROJECT

"Water Level Monitoring and Pump Controller"

STUDENT'S NAME : Malik ALHASAN
STUDENT'S NUMBER : 123456789

STUDENT'S NAME : ----------
STUDENT'S NUMBER : 123456789

STUDENT'S NAME : ----------
STUDENT'S NUMBER : 123456789

PROGRAMMING	CALCULUS II	PHYSICS II	CIRCUIT THEORY I
PROF. DR. Celal ALAGÖZ	ASST. PROF. DR. Semra ZEREN	PROF. DR. Sabit HOROZ	ASST. PROF. Şükrü ÜNVER

2 June 2025

CONTENTS

1 PREFACE

This project has provided us with a valuable opportunity to apply the theoretical knowledge we gained throughout the semester to a real-world problem. By integrating concepts from Physics II, Calculus II, Circuit Theory I, and Programming II, we were able to design an automated water level control system, which allowed us to see how different engineering disciplines can come together to solve a practical challenge.

The idea of creating a system that automatically monitors water levels and controls a pump was particularly appealing to us, as it not only had practical applications but also provided an engaging learning experience. Despite having limited experience with sensor integration and automation at the outset, this project enabled us to develop hands-on skills in microcontroller programming, circuit design, and simulation-based testing.

One of the most challenging aspects of the project was simulating the system in Proteus, where we used virtual components to mimic sensor behavior. Adjusting the system's response to changes in input signals and ensuring the circuit operated correctly was a rewarding challenge. It deepened our understanding of circuit behavior and enhanced our problem-solving abilities.

We would like to express our gratitude to our instructors and teaching assistants, whose guidance and feedback were invaluable throughout the process. Their support helped us move from initial concepts to a fully functional system, and this experience allowed us to see how theoretical knowledge can be applied in real-world situations.

This project has not only improved our technical skills but also strengthened our confidence in developing practical solutions to real-life engineering problems.

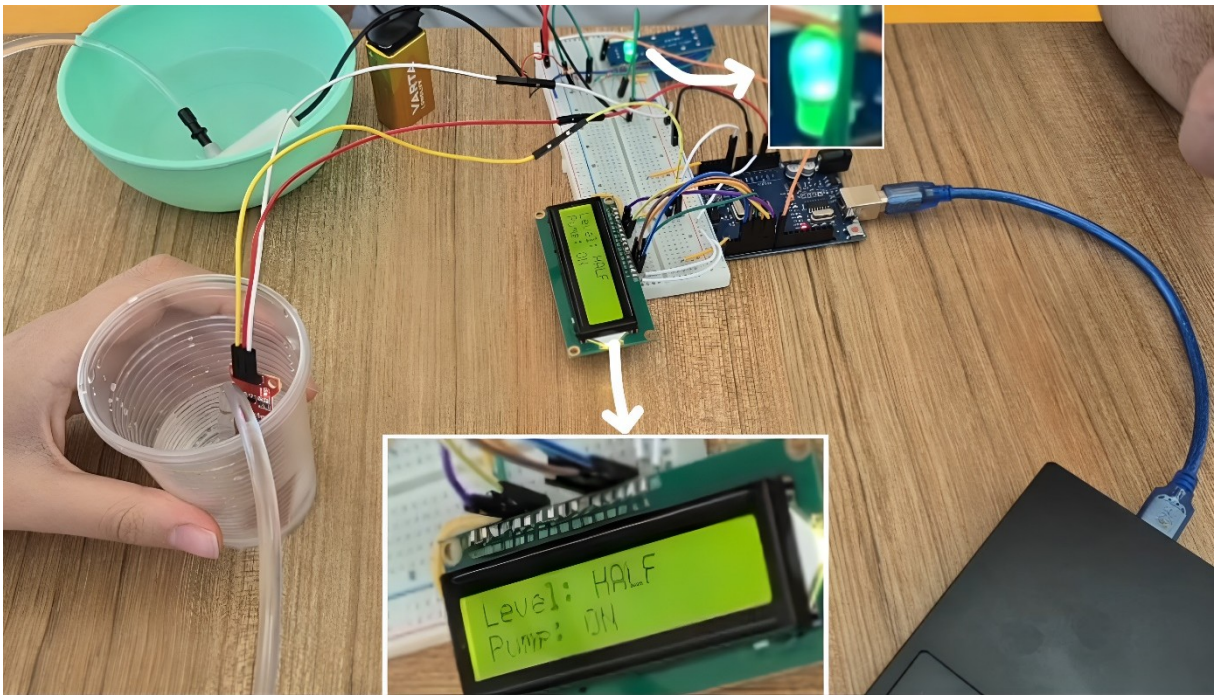
2 INTRODUCTION

In everyday life, the controlled use of water is crucial. Failing to monitor water levels regularly can lead to both wastage and shortages. To address this issue, automated level control systems have been developed, which save time and help the system operate more safely.

In this project, a system was designed to automatically measure water levels and operate a pump when necessary. Based on an Arduino platform, the system tracks the water level using a sensor and controls the pump via a relay. Additionally, the water level and the pump's status are displayed to the user on an LCD screen.

Since the project was carried out individually, the experimental setup was tested with available materials such as a cup and plastic bottle, simulating the water tank. The behaviour of the sensor and the pump was observed, and the system's logic was successfully tested. Furthermore, digital tests were conducted through the Proteus simulation program, where different water levels were simulated using a VPULSE signal.

This project provided an opportunity to practice various technical skills, including Arduino programming, sensor reading, relay control, and creating a user interface with an LCD screen. Moreover, it allowed us to combine knowledge from different courses to solve a real-world problem effectively.



Picture 1 : Test Run the Project

3 MATERIALS USED IN THE PROJECT

After searching through videos and articles, the necessary components for the project were identified. The following materials were found to be needed:

- Ardunio UNO R3 Klon (USB Chip CH340)
- Resistors
- LED
- Water Level / Rain Sensor
- 1 Channel 5V Relay Module
- Mini Submersible Water Pump
- LCD Display
- Breadboard
- Cables
- Batteries (9V)

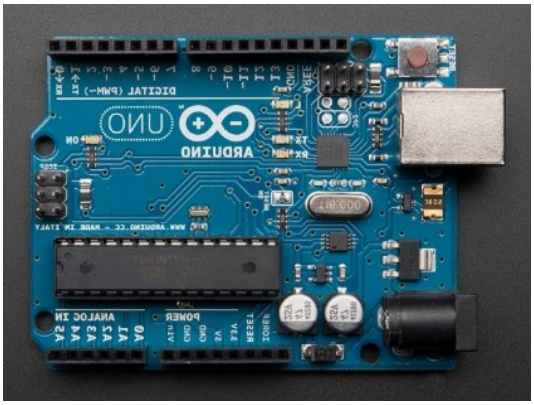
4 FUNCTIONS OF MATERIALS

4.1 Arduino UNO R3 Klon (USB Chip CH340)

Task: The Arduino UNO acts as the brain of the entire system. It reads, processes, and interprets data from sensors and controls actuators like relays and pumps. It also manages LED indicators and runs safety algorithms.

Why Is It Necessary:

- It enables communication between all components.
- Processes sensor data and applies control logic.
- Safely triggers high-power components (via relay).
- Flexible for programming, open-source, and easily programmable with Arduino
- IDE.

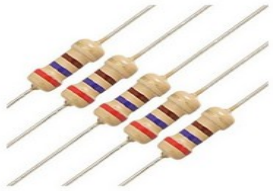


Picture 2 : Ardunio UNO R3 Klon

4.2 Resistors

Task: The resistor is used to limit the current in the circuit and protect the components.

It helps parts like the sensor, relay, and LCD screen work properly without getting damaged.



Picture 3 : Resistors

4.3 Water Level / Rain Sensor

Task: This sensor is used to detect the water level in a tank. It operates based on electrical conductivity. As water touches the sensor's tracks, the output signal changes, which can be read by the Arduino.

Why Is It Necessary:

- Fulfills the primary goal of the experiment: "monitoring water level".
- Fully compatible with Arduino and provides analog output.
- Inexpensive, widely available, and sufficient for basic projects.
- Enables pump control based on tank fullness.

Picture 4 : Rain Sensor

4.4 Channel 5V Relay Module

Task: The relay module allows control of high-current loads (such as a DC water pump) using a low-current signal from a microcontroller like Arduino. When the relay is inactive, the pump is off; when triggered, the pump turns on.

Why Is It Necessary:

- Required to safely operate high-current devices that the Arduino cannot drive directly.
- Components like pumps that draw 400mA or more must not be directly connected to Arduino.

- The relay isolates and protects the circuit.
- A single channel is sufficient for controlling one pump in this project.

Picture 5 : Channel 5V Relay

4.5 Mini Submersible Water Pump

Task: This DC water pump is used to transfer water from one tank to another or to maintain a specific water level. In the experiment, it automatically activates when the water level drops and helps to refill the tank.

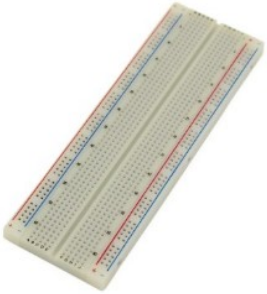
Why Is It Necessary:

- It is the physical actuator that responds to the experiment's logic: activates when
- water level is low.
- Can be safely switched via the relay.
- Compact size makes it suitable for small containers.
- IP68 waterproof rating allows safe operation in liquid environments.

Picture 6 : Mini Submersible Water Pump Module

4.6 Breadboard

Task: The breadboard allows you to build circuits without soldering. It lets you temporarily connect sensors, LEDs, resistors, and other components to test your setup. It's essential for prototyping and experimentation.



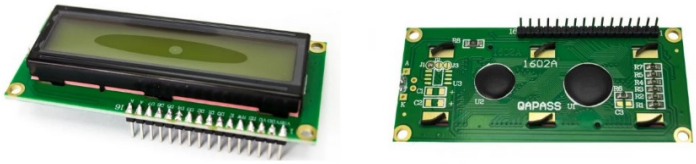
Picture 7 : Breadbord

4.7 LCD Display

Task: This LCD communicates with the Arduino to display textual information to the user. Messages like “Water Level: 50%” or “Pump: ON” can be shown. With 2 lines and 16 characters, it provides concise but sufficient data display.

Why Is It Necessary:

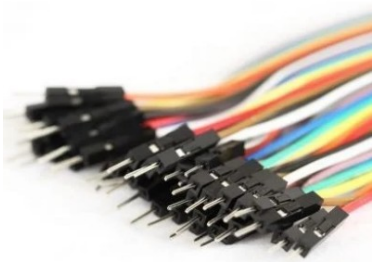
- Goes beyond LED summary by offering detailed status updates.
- Displays real-time system messages via text.
- Comes pre-soldered with headers – ready to use with no soldering required.
- Enhances user interface in both testing and final implementation phases.



Picture 8 : LCD Display

4.8 Cables

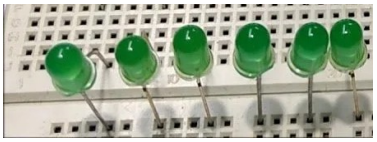
Task: Jumper wires are used to make connections between the breadboard and development boards like Arduino. With male-to-male connectors, they are ideal for plugging directly into both breadboards and module pins.



Picture 9 : Cables

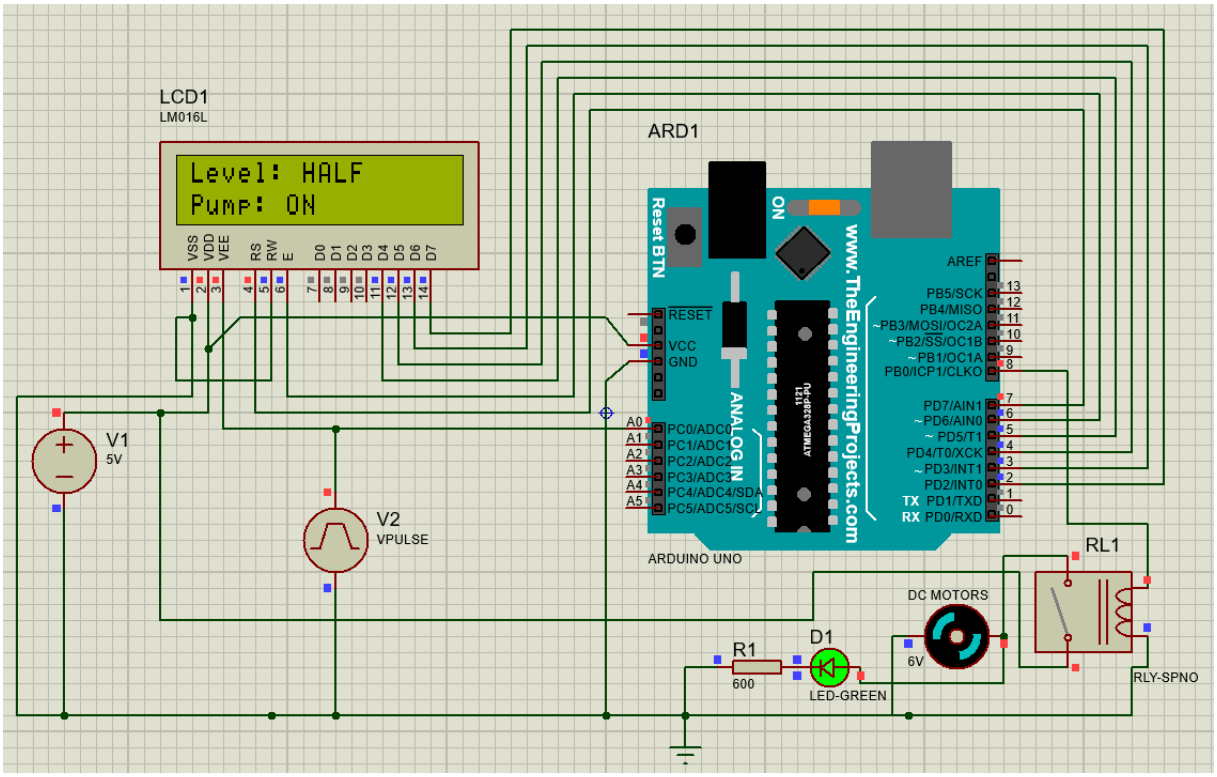
4.9 LED

Task: Led for know if the water pump work or not.



Picture 10 : LED

5 SYSTEM SIMULATION ON PROTEUS



Picture 11 : Circuit Designed On Proteus

5.1 Components Used

- Arduino Uno (ARD1)
- 16x2 LCD Display (LCD1)
- Relay Module (RL1)
- DC Motor (Water Pump)
- Green LED (D1)
- Resistor (R1 - 600 ohms)
- Vcc and GND power supply (V1 = 5V)
- Virtual Sensor (V2 - Simulated with VPULSE)

5.2 Working Principle of the Circuit

This water level monitoring and control circuit uses an **Arduino Uno** (ARD1) to read data from a **water level sensor**, simulate inputs using **VPULSE**, and control a **water pump** through a **relay module**, with status information displayed on an **LCD screen**.

5.2.1 Sensor Input (V2 - VPULSE)

- The **VPULSE generator** simulates a changing analog voltage, representing the water level from a sensor.

- This signal is fed into **Analog Pin A0** of the Arduino.
- In a real system, this would come from a water level sensor (like a rain or float sensor) that changes voltage based on the water level.

5.2.2 Analog Reading and Conversion

- The Arduino reads this analog voltage using `analogRead(A0)`.
- The analog value (ranging from 0 to 1023) is mapped or scaled to represent the **percentage of water level**, e.g., 0 (empty) to 100% (full).
- The program categorizes this into levels such as:
    - EMPTY (0)
    - LOW (0–25%)
    - HALF (25–75%)
    - HIGH (75–100%)

5.2.3 Decision Logic and Pump Control

- The code compares the water level against predefined thresholds:
    - If the level is **below the "HIGH" threshold**, the **pump is turned ON** to fill the tank.
    - If the level reaches or exceeds the threshold, the **pump is turned OFF**.
- The **Relay Module (RL1)** is controlled by a digital pin from the Arduino (D8), which acts like a switch for the **DC Motor (Pump)**.

5.2.4 Visual Indicators

- A **Green LED (D1)** connected with **Resistor R1 (600 ohms)** serves as a basic indicator, showing pump status.
- The **16x2 LCD (LCD1)** is used to give real-time feedback:
    - It shows the water level state (e.g., “Level: HALF”).
    - It shows the pump status (e.g., “Pump: ON”).

5.2.5 Power Supply

- The circuit uses a **5V power supply (V1)**, which powers the Arduino and components.
- The simulation assumes a 5V pump for simplicity, but real pumps need higher voltages and external power supplies.

6 C++ CODE EXPLANATİON

This Arduino code is written to control a water pump based on water level readings and to display the status on a 16x2 LCD screen. It uses a water level sensor connected to the analog input pin **A0** and a **relay module** connected to **digital pin 8** to switch the pump ON or OFF.

6.1 LCD Setup

```
#include <LiquidCrystal.h>

// LCD pin configuration: RS, E, D4, D5, D6, D7
LiquidCrystal lcd(7, 6, 5, 4, 3, 2);
```

- The LiquidCrystal library is used to control the 16x2 LCD.
- The LCD is connected to **digital pins 2 to 7**, with the configuration: RS (7), E (6), D4-D7 (5, 4, 3, 2).

6.2 Pin Assignments

```
// Pin assignments
const int waterSensorPin = A0;
const int controlPin = 8;
```

- waterSensorPin: Analog pin **A0** reads the water level.
- controlPin: Digital pin **8** controls the relay, which switches the **pump** ON or OFF.

6.3 Setup Function

```
void setup() {
  lcd.begin(16, 2);           // Initialize the 16x2 LCD
  pinMode(controlPin, OUTPUT);// Set pump control pin as output

  // Display welcome message
  lcd.setCursor(3, 0);
  lcd.print("Water Level");
  lcd.setCursor(5, 1);
  lcd.print("System");
  delay(2000);                // Wait for 2 seconds
  lcd.clear();                // Clear LCD
}
```

- Initializes the LCD and sets the **pump control pin** as output.
- Displays a short welcome message before beginning sensor monitoring.

6.4 Main Loop Function

```
void loop() {
  int sensorValue = analogRead(waterSensorPin);
  sensorValue = constrain(sensorValue, 10, 250);
  int levelPercent = map(sensorValue, 10, 250, 0, 100);
```

- The **sensor value** is read from A0 and **constrained** between 10 and 250 to filter noise or out-of-range values.
- It is then **mapped** to a 0–100% water level range.

```
  lcd.setCursor(0, 0);
  lcd.print("Level: ");
```

- Sets the cursor on the LCD to the top row for displaying the **water level status**.

6.5 Water Level Status

```
  // Display level description
  if (levelPercent >= 90) {
    lcd.print("HIGH   ");
  } else if (levelPercent >= 75) {
    lcd.print("HALF   ");
  } else if (levelPercent >= 25) {
    lcd.print("LOW    ");
  } else {
    lcd.print("EMPTY  ");
  }
```

- Displays water level condition as: **HIGH**, **HALF**, **LOW**, or **EMPTY**, based on the percentage.

6.6 Pump Control Logic

```
  // Pump logic: ON until FULL
  if (levelPercent >= 95) {
    digitalWrite(controlPin, HIGH);    // Turn pump OFF
    lcd.setCursor(0, 1);
    lcd.print("Pump: OFF    ");
  } else {
    digitalWrite(controlPin, LOW);     // Turn pump ON
    lcd.setCursor(0, 1);
    lcd.print("Pump: ON     ");
  }

  delay(500);
}
```

- If the level reaches or exceeds **95%**, the pump is **turned OFF** to avoid overflow.
- If it is **below 95%**, the pump is **turned ON**.
- The **LCD** shows the pump status accordingly.
- A **delay of 500ms** is used to update the reading twice per second.

6.7 Summary

This code efficiently automates a simple water management system by:

- Monitoring water levels,
- Controlling a pump using a relay,
- And giving real-time feedback via an LCD.

It ensures the system responds in real time to avoid overflow while keeping the user informed.

7 CONCLUSION

7.1 Project Summary

The "Water Level Monitoring and Pump Controller" project successfully demonstrated how theoretical knowledge from courses such as Programming, Circuit Theory, and Physics can be applied to build a practical and functional automated system. The goal of the project was to monitor the water level using a sensor and control a submersible water pump through a relay module, with live feedback displayed on an LCD screen.

7.2 Implementation and Testing

The system was developed and tested both in simulation using Proteus and in real-life hardware. While the Proteus simulation proceeded smoothly, the real-life implementation revealed valuable challenges that deepened our understanding of hardware behavior.

7.3 Challenges and Solutions

One major issue involved the water level sensor: although initially assumed to produce values between 0 and 1023, we discovered that our specific sensor output ranged only from 0 to 250. After reprogramming the Arduino code to reflect these actual values, the system operated correctly.

Another key challenge was related to power management. In the Proteus environment, a single 5V power source was sufficient for the entire system. However, during real-world testing, we encountered instability due to the high current draw of the water pump (approximately 0.5A). This led us to separate the power supply into two sources—one for the Arduino and control circuit, and another dedicated to the pump. Additionally, we observed that the sensor gave incorrect readings when powered at 5V and functioned properly only at 3.3V, which was resolved by adjusting the power supply accordingly.

7.4 Skills and Learning Outcomes

Through this project, we gained hands-on experience in sensor calibration, Arduino programming, circuit design, LCD interfacing, and simulation tools. It also taught us how to troubleshoot real hardware problems, manage power distribution, and adapt code logic based on sensor behavior. All group members collaborated actively and contributed equally to the research, testing, and final implementation of the project.

7.5 Future Improvements

Looking ahead, the system could be improved by integrating wireless communication (e.g., Bluetooth or Wi-Fi) for remote monitoring, adding multiple sensors points for better accuracy, or developing a mobile application interface. These enhancements could make the system more scalable and user-friendly for real-world applications like irrigation systems or water tank automation.

8 REFERANCE

1. **Arduino Official Documentation**
   Arduino. (n.d.). *Arduino Documentation*. Retrieved from https://docs.arduino.cc
2. **Proteus Simulation Software**
   Labcenter Electronics. (n.d.). *Proteus Design Suite*. Retrieved from https://www.labcenter.com
3. **16x2 LCD Display with Arduino Tutorial**
   Dejan. (n.d.). *Arduino 16×2 LCD Tutorial – Everything You Need to Know*. HowToMechatronics. Retrieved from https://howtomechatronics.com/tutorials/arduino/lcd-tutorial/
4. **Relay Module with Arduino Guide**
   Circuit Digest. (n.d.). *Understanding How a Single Channel Relay Module Works and Interface with Arduino*. Retrieved from https://circuitdigest.com/microcontroller-projects/interface-single-channel-relay-module-with-arduino
5. **Submersible Pump: Working Principles and Applications**
   Linquip. (n.d.). *Submersible Pump: Working Principles, Function & Diagram*. Retrieved from https://www.linquip.com/industrial-directories/431/submersible-pumps
6. **Water Level Sensor with Arduino Tutorial**
   Last Minute Engineers. (n.d.). *Water Level Sensor with Arduino*. Retrieved from https://lastminuteengineers.com/water-level-sensor-arduino-tutorial/
7. **General Electronics Knowledge**
   Horowitz, P., & Hill, W. (2015). *The Art of Electronics* (3rd ed.). Cambridge University Press.